Aluminium Price Forecast API

A FastAPI service that forecasts aluminium part prices for the next 12 months. Each month's price is built from LME, Midwest premium, PPI and CNG inputs. The base price changes only once per quarter, in the last month of the quarter, and small changes are held steady by a ±2% deadband. LME and PPI inputs can themselves be auto-forecast with SARIMA models before every batch run, and an optional AOP (Annual Operating Plan) quantity can be applied to turn unit prices into total dollar exposure per part per month.

Table of Contents

1. Project Overview
2. Forecasting Logic
3. Dynamic LME & PPI Forecasting (SARIMA)
4. AOP (Annual Operating Plan)
5. Repository Structure
6. File-by-File Guide
7. Data Layer
8. API Endpoints
9. Getting Started
10. How to Swap a Data Source
11. Excel File Format
12. Frontend Integration
13. Known Gaps and Legacy Files

---

1. Project Overview

The service predicts the price of an aluminium part for the **next 12 calendar months** (or 13 when the current month is included). The current month comes from the **UTC system clock**.

Key concepts

- **Part Number + Tier 1** form the unique key for a part. The same part number can appear under several Tier 1 suppliers (e.g. "Kadon Aerospace", "QCC, LLC", "NA") with different weights and prices.
- **PWt** is the part weight in lbs, fixed per Part Number + Tier 1.

- **Base price** is always read from the Parts sheet of `aluminium_data.xlsx`. It cannot be overridden through the API.
- **Market data** (LME, Midwest, PPI) is read from `aluminium_data.xlsx` and cached in memory.
- **CNG_Q and CNG_Q-1** ($/lb) are either supplied once for the whole batch in the request, or read per part from the uploaded Excel file (§3.4 of the old flow is now covered in §4/§6).
- **Every month is calculated independently** from its own market data, using a base price chosen by the quarterly base-price rule (§2.3). A predicted price is not simply rolled into the next month.
- **LME and PPI can be auto-forecast.** If the workbook has `lme_input` / `ppi_input` sheets of historical actuals, the batch endpoint fits a SARIMA model and refreshes the `LME` / `PPI` sheets with predicted values before every run (§3).
- **AOP (Annual Operating Plan)**: if the workbook has an AOP sheet of planned quantities per part per month, the batch output can show a Total AOP ($) value (quantity × predicted price) per part per month, plus a dedicated summary sheet (§4).

---

2. Forecasting Logic

2.1 Core formula (applied to every forecast month)

```
raw_price = P_current
            + [ (AMS_Q − AMS_Q-1) × PWt ]
            + [ PPI_Factor × P_current ]
```

Variable	Definition
`MC_Q`	`(LME + Midwest) + MC_OFFSET` for the **month being predicted**
`MC_Q-1`	Same expression for the previous quarter (see 2.2)
`AMS_Q`	`(MC_Q × DF_c) + CNG_Q`
`AMS_Q-1`	`(MC_Q-1 × DF_c) + CNG_Q-1`
`PPI_Q`	PPI index of the **month being predicted**
`PPI_Q-1`	PPI of the previous quarter (see 2.2)
`PPI_Factor`	`(PPI_Q − PPI_Q-1) / PPI_Q-1`
`CNG_Q`, `CNG_Q-1`	Supplied in the request, or read per part from the batch upload (§4/§6)
`DF_c`	`1.44` (`DF_C` in config)

Variable	Definition
`MC_OFFSET`	`0.558883` (config)
`PWt`	Part weight in lbs
`P_current`	Base price for the month being predicted, chosen by the quarterly rule (see 2.3)

> **MC formula:** the current code uses `MC = (LME + Midwest) + 0.558883`. The older `× 0.91 + 1.25` form is **not used**. `MC_MULTIPLIER` still exists in `config.py` but the engine ignores it. The previous version is (`× MC_MULTIPLIER + MC_OFFSET`). So this logic is updated and this is from thapase/srikanth suggestions on 9/30/2026

So This MC_OFFSET is the newly added feature in this version. And this is took from thapase/srikanth requirements on 9/30/2026 ######### CHANGE

2.2 Previous-quarter values (`_Q-1`)

The previous quarter is the calendar quarter before the one containing the predicted month. Both `MC_Q-1` and `PPI_Q-1` follow the same "actual vs projected" rule:

- If the **last month of the previous quarter** is covered by actuals, use the value of **that month**.
- Otherwise use the **average of the three months** of the previous quarter. For MC this averages MC, LME and Midwest.

A month counts as "actual" when `last_month ≤ current_month − PPI_ACTUAL_LAG_MONTHS`. The default lag is **1** in `config.py`, so sitting in October 2026 actuals run through September 2026.

2.3 Base price rule

Rule. It depends on the position of the forecast month inside its calendar quarter (Jan–Mar, Apr–Jun, Jul–Sep, Oct–Dec):

- **1st and 2nd month of a quarter** use `active_base` (the price carried over from the previous quarter's rule).
- **3rd (last) month of a quarter** uses the final price that was just calculated for that same quarter's 1st month — not `active_base`.

How the active base moves. The engine keeps two values while it loops over the months:

- `active_base` starts as the price from the Parts sheet.
- `pending_first_month_price` holds the final price of the current quarter's 1st month.

```
position = _month_position_in_quarter(forecast_month)       # 1, 2 or 3
if position == 3 and pending_first_month_price is not None:
    current_price = pending_first_month_price               # 3rd month: this
quarter's 1st-month final price
```

```
else:
    current_price = active_base                              # 1st and 2nd month:
active base
...
if position == 1:
    pending_first_month_price = predicted_price              # remember the 1st
month's final price
if position == 3:
    active_base = current_price                              # carry it into the next
quarter's 1st and 2nd months
    pending_first_month_price = None
```

If the 1st month's calculated price is within ±2% of its base, the deadband holds the price, so its final price equals the base and the base does not change.

So This Base price rule is the newly added feature in this version. And this is took from thapase/srikanth requirements on 10/11/2026 ######### CHANGE

2.4 Deadband (±2%)

So This Deadband logic is the newly added feature in this version. And this is took from thapase/srikanth requirements on 9/30/2026 ######### CHANGE

After calculating `raw_price`:

```
if |raw_price − P_current| / P_current ≤ DEADBAND_PCT (0.02):
    predicted_price = P_current        # change too small, price is held
else:
    predicted_price = raw_price
```

The engine computes and returns both `predicted_price` (after the deadband, the "final price") and `predicted_price_without_deadband` (the raw value). The deadband output is what is carried forward as a base price through the quarterly rule (§2.3).

> **Excel column naming note (important):** in the batch output workbook, the monthly-sheet column literally named "**Predicted Price**" holds `predicted_price_without_deadband` (the raw, pre-deadband value), and the column named "**Final Unit Price (with deadband)**" holds `predicted_price` (the final, deadband-adjusted value). This is the opposite of what the names might suggest from the JSON API, where `predicted_price` is always the final (with-deadband) value. See §6 and §11 for the exact column list.

2.5 Forecast window and price month

`include_current_month` (`"YES"` / `"NO"`, default `"NO"`) decides where the forecast starts. The Parts sheet price column header must name the month the prices are for, e.g. `Current Price ($) August 2026`.

`include_current_month`	**Price month needed**	**First forecast month**	**Months**
`"NO"` (default)	**This** month	Next month	12
`"YES"`	**Last** month	This month	13

If the header month does not match, the request fails with a message saying which month the data holds and which is needed.

2.6 Worked example (sitting in Sep 2026, `include_current_month = "NO"`)

The forecast covers Oct 2026 → Sep 2027. `F(month)` means the final price (after the deadband) of that month, and "Excel" is the price from the Parts sheet.

Month	**Position in quarter**	`P_current` **used**	**Carried forward as a base?**
Oct 2026	1st	Excel	Yes: `F(Oct 2026)` is the base for Dec 2026, Jan 2027 and Feb 2027
Nov 2026	2nd	Excel	No
Dec 2026	3rd (last)	`F(Oct 2026)`	No
Jan 2027	1st	`F(Oct 2026)`	Yes: `F(Jan 2027)` is the base for Mar, Apr and May 2027
Feb 2027	2nd	`F(Oct 2026)`	No
Mar 2027	3rd (last)	`F(Jan 2027)`	No
Apr 2027	1st	`F(Jan 2027)`	Yes: `F(Apr 2027)` is the base for Jun, Jul and Aug 2027
May 2027	2nd	`F(Jan 2027)`	No
Jun 2027	3rd (last)	`F(Apr 2027)`	No
Jul 2027	1st	`F(Apr 2027)`	Yes: `F(Jul 2027)` is the base for Sep 2027 onwards

Month	Position in quarter	`P_current` used	Carried forward as a base?
Aug 2027	2nd	`F(Apr 2027)`	No
Sep 2027	3rd (last)	`F(Jul 2027)`	No

Each month computes its own `MC_Q`, `PPI_Q`, `AMS_Q` and `PPI_Factor`, then applies the core formula and the deadband on its own `P_current`. The **Base Price Summary** sheet in the batch output shows the `P_current` column above for every part.

2.7 Price-change breakdown

The single-part endpoint also returns a decomposition of `raw_price − P_current` into four exact factors:

```
lme_effect     = (lme_q − lme_q-1) × DF_c × PWt
midwest_effect = (midwest_q − midwest_q-1) × DF_c × PWt
cng_effect     = (cng_q − cng_q-1) × PWt
ppi_effect     = ppi_factor × P_current
```

They sum to `total_change` (`sum_check` should be 0). The breakdown uses the price **before** the deadband so `sum_check` stays 0. It is not produced by the batch endpoint.

2.8 Rounding of decimal values

Principle: every calculation runs at full floating-point precision. Rounding is applied only when a value is placed in the API response or in the Excel output. Rounded values are never fed back into later calculations, so the deadband check and the base price carried forward (§2.3, §2.4) always use the unrounded numbers.

API response (JSON), decimals kept:

Field	Decimals
`predicted_price`, `predicted_price_without_deadband`, `base_price_used`	4 (engine-level `round(..., 4)`)
`mc_q`, `lme_q`, `midwest_q`, `cng_q`, `ams_q` and their `_q-1` versions	6
`ams_delta`	6
`ppi_q`, `ppi_q-1`	4

Field	Decimals
`ppi_factor`	8
`pwt_lbs`, `df_c`, top-level `base_price`	Not rounded (returned as stored)

Batch Excel output, decimals kept (and now actually stored rounded, not just displayed rounded):

Column / sheet	Decimals	Cell format	Underlying value
Base Price (Summary & monthly sheets)	2	`$#,##0.00`	`round(base_price_used, 2)`
"Predicted Price" (monthly sheet)	2	`$#,##0.00`	`round(predicted_price_without_deadband, 2)` — **raw, pre-deadband** (see note in §2.4)
"Final Unit Price (with deadband)" (monthly sheet)	2	`$#,##0.00`	`round(predicted_price, 2)` — the actual final price
`MC_Q`, `MC_Q-1`, `AMS_Q`, `AMS_Q-1`, `AMS Delta`	2	`$0.00`	rounded
`PPI_Q`, `PPI_Q-1`, `PPI Factor`	1	`0.0`	rounded
`CNG_Q`, `CNG_Q-1`, `DF_c`	Not rounded	`General`	shown exactly as stored (e.g. `0.97`, not `0.9700`)
Weight	Not rounded (shown with 2 decimals)	`0.00`	

Column / sheet	Decimals	Cell format	Underlying value
Summary and Base Price Summary sheets (all prices)	2	`$#,##0.00`	rounded
Total AOP (monthly sheet and Total AOP Summary)	2	`$#,##0.00`	`round(unit_price × aop_qty, 2)`, blank if no AOP qty for that part/month

---

3. Dynamic LME & PPI Forecasting (SARIMA)

`app/services/lme_forecast.py` and `app/services/ppi_forecast.py` let the workbook forecast its **own** LME and PPI inputs with SARIMA models, instead of requiring someone to type predicted values into the `LME` / `PPI` sheets by hand.

3.1 What each one does

	LME (`lme_forecast.py`)	**PPI** (`ppi_forecast.py`)
Reads actuals from	`lme_input` sheet	`ppi_input` sheet
Writes predictions into	`LME` sheet (the sheet the engine reads)	`PPI` sheet (the sheet the engine reads)
Model	log-SARIMA(1,1,0)(0,1,1,12)	SARIMA(1,1,1)(1,1,1,12)
Training window	last 72 months of `lme_input`	last 67 months of `ppi_input`
Forecast horizon	12 months	12 months
Unit conversion	divides by `2204.6226` ($/MT → $/lb)	none
Outlier handling	IQR + Winsorization (non-cascading)	IQR + Winsorization
Accuracy check	12-month backtest, `100 − MAPE`, written to an `Accuracy (%)` column (row 2) of the `LME` sheet	same, written to the `PPI` sheet

3.2 When it runs

`app/services/batch_forecast.py` calls `refresh_ppi_forecast()` and `refresh_lme_forecast()` **once at the start of every batch run** (both the shared-CNG path and the per-row-CNG path — see §4/§6), so the batch always forecasts against the latest predicted LME and PPI values. This is not currently wired into the single-part JSON endpoint.

3.3 Safe-by-design behaviour

Both modules are built to never break a batch run:

- No `lme_input` / `ppi_input` sheet → does nothing (old behaviour, unchanged `LME` / `PPI` sheets).
- Input data unchanged since the last refresh (same row count, same latest month, same checksum of the data) → the model is **not** re-fitted.
- Predicted values identical to what's already in the sheet → the workbook is **not** re-saved.
- Any error (file open in Excel, `statsmodels` missing, a gap in the month sequence, SARIMA convergence failure, etc.) is only **logged** — the batch continues using the `LME` / `PPI` sheet as it currently stands.

3.4 Side effects to be aware of

- A successful refresh **writes to `aluminium_data.xlsx` on disk** (new or updated rows in the `LME` / `PPI` sheets, plus the `Accuracy (%)` column). `ppi_forecast.py` makes a timestamped backup copy first (`BACKUP_BEFORE_UPDATE = True`, keeps the newest 2); `lme_forecast.py` currently does not (`BACKUP_BEFORE_UPDATE = False`).
- After a successful write, the in-memory Excel cache (`excel_store`) is invalidated, so the very same batch run reads the freshly predicted LME/PPI values.
- `statsmodels` is imported lazily inside `forecast_lme()` / `forecast_ppi()`, so the app still starts if it isn't installed — only the refresh step fails (and is logged) in that case. It must be added to `requirements.txt` (see §9).
- Dates in `lme_input` / `ppi_input` and in the `LME` / `PPI` sheets can be written in almost any common format (Excel date, `2026-10`, `Oct 2026`, `Oct'26`, `10/2026`, `01-Oct-26`, an Excel serial number, etc.) — both modules share a flexible `_parse_month()` / `_month_key()` parser for this.

---

4. AOP (Annual Operating Plan)

AOP lets the batch output translate a **unit price forecast** into a **total dollar exposure**, using a planned purchase quantity per part per month.

4.1 Where the quantities come from

`app/data/excel_store.py` exposes `_load_aop_lookup()`, which reads an AOP sheet from the same workbook (Part Number, Tier 1, and one column per month, each header naming a month — mirrors the

layout `forecast_engine.build_aop_lookup()` parses) into a lookup keyed by `(part_number, tier_1, 'YYYY-MM')`. Matching is case-insensitive and whitespace-trimmed. If no tier-specific quantity is found, an item-level entry (empty tier) is used as a fallback.

> The exact AOP sheet name/layout lives in `excel_store.py`, which was not part of this review — confirm the sheet name and header format there if a part's Total AOP keeps coming back blank.

4.2 How a total is computed

```
def compute_total_aop(unit_price: float, aop_qty: Optional[float]) ->
Optional[float]:
    if aop_qty is None:
        return None                      # no AOP qty for this part/month -> blank
cell
    return round(unit_price * aop_qty, 2)
```

`unit_price` here is always `predicted_price` (the **final**, deadband-adjusted price), both in the monthly-sheet "Total AOP" column and in the Total AOP Summary sheet.

4.3 Where it shows up in the batch output

- **Each monthly sheet** gets one extra column, **"Total AOP ($)"**, appended after the normal columns — only when an AOP lookup was found (`aop_lookup` is non-empty). Blank when the part has no AOP quantity for that month.
- **"Total AOP Summary"** sheet (3rd tab, after Summary and Base Price Summary — only created when an AOP lookup is found): one row per (Part Number, Tier 1), one column per forecast month showing **that month's own** Total AOP (not cumulative — same number as the matching cell in that month's own sheet), plus a final **"Total AOP (All Months)"** column that sums every month for that part. A bold **"Grand Total"** row at the bottom sums only the "Total AOP (All Months)" column (the per-month columns are left blank on that row).

4.4 Programmatic use

`ForecastEngine.apply_aop(response, aop_lookup)` is a standalone helper (not used by the batch flow above, but available for other callers) that takes one part's `ForecastResponse` and an AOP lookup, and returns `{year_month: (aop_qty, total_aop_value)}` for that part, again using the final `predicted_price` by default.

---

5. Repository Structure

The code imports modules as `app.core...`, `app.data...`, etc. The files below are therefore expected in this layout. In the uploaded archive they are all in one flat folder, so they need to be placed into these folders before the server will start.

```
project/
├── main.py                      # FastAPI entry point
├── aluminium_data.xlsx          # Master data (Parts, LME, Midwest, PPI, lme_input,
ppi_input, AOP sheets)
├── input.xlsx                   # Sample batch input (Part Number, Tier 1[, CNG_Q,
CNG_Q-1])
├── requirements.txt
├── approach.txt                 # Original problem statement (older logic, see §13)
│
├── app/
│   ├── api/v1/
│   │   ├── router.py                  # Registers endpoint routers
│   │   └── endpoints/
│   │       ├── forecast_excel.py      # Single-part + /parts + /reload
│   │       ├── forecast_batch.py      # Batch upload/download
│   │       └── forecast.py            # LEGACY, not registered (§13)
│   ├── core/
│   │   ├── config.py                  # Settings and constants
│   │   └── logging_config.py
│   ├── data/
│   │   ├── base.py                    # Abstract repository interfaces
│   │   ├── excel_store.py             # Active data source (incl.
_load_aop_lookup)
│   │   └── hardcoded_store.py         # Legacy in-code data (tests only)
│   ├── models/
│   │   ├── request.py                 # Endpoints Requests
│   │   └── response.py                # Endpoints Responses
│   └── services/
│       ├── forecast_engine.py         # Current formula logic + AOP helpers
│       ├── forecast_engine_v1.py      # Previous version (× 0.91 + offset),
reference only
│       ├── lme_forecast.py            # NEW: SARIMA auto-forecast of the LME sheet
│       ├── ppi_forecast.py            # NEW: SARIMA auto-forecast of the PPI sheet
│       └── batch_forecast.py          # Batch orchestration + output workbook (+
AOP summary)
│
└── tests/
    └── test_forecast_engine.py        # For testing (outdated — see §13)
```

> Every folder inside `app/` needs an empty `__init__.py`, otherwise you will get a `ModuleNotFoundError`.

---

6. File-by-File Guide

`main.py`

Creates the FastAPI app, configures logging, adds CORS (`allow_origins=["*"]` , tighten for production), registers a global exception handler returning HTTP 500 JSON, mounts `api_v1_router` , and exposes `GET /health` , which returns `{"status": "ok", "version": ...}` . The startup log prints `DF_c` and the horizon.

```
uvicorn main:app --reload --port 8000
```

`app/core/config.py`

`pydantic-settings` class. Every value can be overridden by an environment variable or a `.env` file.

Setting	Default	Meaning
`DF_C`	`1.44`	Conversion factor in AMS
`MC_MULTIPLIER`	`0.91`	**Unused by the current engine**
`MC_OFFSET`	`0.558883`	Added to `(LME + Midwest)` to get MC
`PPI_ACTUAL_LAG_MONTHS`	`1`	Months before the current month that actuals end. Decides actual vs average for `MC_Q-1` and `PPI_Q-1`
`FORECAST_HORIZON_MONTHS`	`12`	Months forecast (13 when the current month is included)
`DEADBAND_PCT`	`0.02`	±2% band within which the base price is retained
`PRICE_LAG_MONTHS`	`2`	**No longer used by the engine.** Replaced by the quarterly base-price rule (§2.3). Safe to remove
`APP_NAME` , `APP_VERSION` , `DEBUG`	—	App metadata and debug logging

`app/core/logging_config.py`

`configure_logging(debug)` sets one log format (`time | level | module:line | message`) and quiets `uvicorn.access` . Other modules just use `logging.getLogger(__name__)` .

`app/data/base.py`

Abstract contracts the engine depends on:

- **`MarketDataRepository`** : `get_lme()` , `get_midwest_premium()` , `get_ppi()` . Each takes `YYYY-MM` and returns `float | None` .

- `PartRepository`: `get_part_weight(part_number, tier_1)`, `get_base_price(part_number, tier_1, year_month)`, plus an optional `get_price_month()` used to give clearer price-month errors.

`app/data/excel_store.py` (active data source)

Loads the `Parts`, `LME`, `Midwest` and `PPI` sheets with `lru_cache`, plus `_load_aop_lookup()` for the AOP sheet (§4.1).

- Everything is read as text with `keep_default_na=False`, so the supplier name `"NA"` is not turned into NaN.
- `ExcelMarketDataRepository` looks up month → value for LME, Midwest and PPI.
- `ExcelPartRepository` matches rows on (Part Number, Tier 1), reading columns **by position** (0 part, 1 tier, 2 weight, 3 price).
- `_parse_price_month()` reads the month from the price column header (`Aug 2026`, `August 2026`, `2026-08`, `08/2026`, or a date cell).
- `get_base_price()` returns a price **only if** the requested month equals the header month.
- `get_all_parts()` feeds `GET /parts`. `_invalidate_cache()` backs `POST /reload`, and is also called automatically whenever `lme_forecast.py` / `ppi_forecast.py` successfully update the `LME` / `PPI` sheets.
- The file path defaults to `aluminium_data.xlsx` two levels above `app/data/`; override it with `ALUMINIUM_EXCEL_PATH`.

`app/data/hardcoded_store.py`

Original in-code data set (July 2025 onwards) kept for tests and as a pattern for new adapters. Not used by active endpoints.

`app/services/forecast_engine.py` (core logic)

`ForecastEngine(market_repo, part_repo).forecast(part_number, tier_1, cng_q, cng_q_1, include_breakdown=False, include_current_month=False)`

Steps:

1. Read the current UTC month and decide the price month (this month, or last month when `include_current_month` is true).
2. Look up `PWt` (error "Part not found" if missing), then the base price (error naming the expected month if the header month differs).
3. Loop over 12 months (13 if the current month is included), starting the month after the price month. For each month:
    1. Pick `P_current` using the quarterly base-price rule (§2.3).
    2. Build the `QuarterContext` (MC, PPI, AMS and previous-quarter values).
    3. Compute `raw_price`, apply the deadband, and optionally the factor breakdown.

4. Store the result, flagged `is_data_projected = (step ≥ 1)`.
5. Update the quarter state: after a 1st month, remember its final price; after a 3rd month, make the base used in that month the new active base.

4. Return a `ForecastResponse`.

Also in this file (AOP, §4): `build_aop_lookup(rows)`, `compute_total_aop(unit_price, aop_qty)`, `_aop_month_key(header)`, `_aop_part_key(value)`, and `ForecastEngine.apply_aop(response, aop_lookup)`.

Helper functions: `_quarter_of_month`, `_quarter_months`, `_last_month_of_quarter`, `_prev_quarter`, `_month_position_in_quarter`, `_shift_month`, `_compute_mc`, `_ppi_prev_quarter`, `_compute_quarter_context`, `_compute_price_breakdown`, `_price_month_error`.

`app/services/lme_forecast.py` (NEW)

SARIMA auto-forecast of the `LME` sheet from `lme_input` actuals. Entry point: `refresh_lme_forecast(path, force=False)`, called by `batch_forecast.py` before every batch run. See §3 for the full behaviour. Never raises — failures are logged and the existing `LME` sheet is used as-is.

`app/services/ppi_forecast.py` (NEW)

SARIMA auto-forecast of the `PPI` sheet from `ppi_input` actuals. Entry point: `refresh_ppi_forecast(path, force=False)`, called by `batch_forecast.py` before every batch run. See §3 for the full behaviour. Never raises — failures are logged and the existing `PPI` sheet is used as-is.

`app/services/batch_forecast.py`

- **`read_parts_from_upload(file_bytes)`**: finds `Part Number` and `Tier 1` columns (case-insensitive), de-duplicates while keeping order, returns `(part, tier)` tuples. Used by the shared-CNG batch flow.
- **`read_parts_with_cng_from_upload(file_bytes)`** (NEW): same, plus reads per-row `CNG_Q` and `CNG_Q-1` columns (several header spellings accepted) directly from the upload, so each part can have its own CNG values instead of one shared pair for the whole batch.
- **`build_forecast_workbook(pairs, engine, cng_q, cng_q_1, include_current_month)`**: shared-CNG path. Calls `refresh_ppi_forecast()` / `refresh_lme_forecast()` first, then runs the engine for each pair. A failing part gets a red **ERROR** row rather than stopping the batch. An error is raised only if every part fails.
- **`build_forecast_workbook_from_rows(part_rows, engine, include_current_month)`** (NEW): per-row-CNG path, same refresh + error handling, but each part uses its own CNG values from `read_parts_with_cng_from_upload`.
- **`build_preview_from_rows(part_rows, engine, include_current_month)`** (NEW): returns a JSON-serialisable preview (summary, base price summary, and one entry per monthly sheet) of the

same data that goes into the Excel workbook, for on-screen display before downloading. Whether/at what route this is exposed depends on `forecast_batch.py`, which was not reviewed here.

Output workbook sheets (`_build_workbook_from_results`):

1. **Summary**: predicted (final, with-deadband) price per part per month.
2. **Base Price Summary**: the `P_current` used per part per month. And this sheet is updated from thapase/srikanth srequirements.
3. **Total AOP Summary** (NEW, only when an AOP lookup is found — §4.3): Total AOP per part per month, plus an all-months total per part and a grand total.
4. **One sheet per forecast month** (12, or 13 with the current month), with these columns (see §2.8 for the exact rounding of each):

`Part Number, Tier 1, Weight, Base Price, Quarter (Current), Quarter (Previous), MC_Q, MC_Q-1, PPI_Q, PPI_Q-1, PPI Factor, CNG_Q, CNG_Q-1, AMS_Q, AMS_Q-1, AMS Delta, DF_c, Predicted Price (= raw, without deadband), Final Unit Price with deadband (= the actual final price)[, Total AOP]`

> See the note in §2.4 — "Predicted Price" and "Final Unit Price (with deadband)" are intentionally the reverse of what their names might suggest relative to the JSON field names.

`app/models/request.py`

`ForecastRequest`:

```
{
  "part_number": "09-0052-003",
  "tier_1": "NA",
  "cng_q": 0.97,
  "cng_q-1": 0.95,
  "include_current_month": "NO"
}
```

`cng_q` and `cng_q-1` (alias for `cng_q_1`) are **required** on the single-part and shared-CNG batch paths. `include_current_month` accepts `YES` / `NO` in any case.

`app/models/response.py`

- **`QuarterContext`**: `quarter_label`, `mc_q`, `lme_q`, `midwest_q`, `ppi_q`, `cng_q`, `ams_q`, the previous-quarter equivalents (serialised as `mc_q-1`, `lme_q-1`, etc.), `ppi_factor`, `ams_delta`.
- **`PriceFactorBreakdown`** / **`PriceChangeBreakdown`**: dollar and % effect of LME, Midwest, CNG and PPI, plus `sum_check`.
- **`MonthForecast`**: `year_month`, `month_label`, `predicted_price` (final, with deadband), `predicted_price_without_deadband` (raw), `base_price_used`, `pwt`, `df_c`, `quarter_context`

(required on every month, since the quarterly rule — unlike the earlier 50-day-lag design — always computes a fresh context), `price_change_breakdown` (single-part endpoint only), `is_data_projected`.

- `ForecastResponse`: `part_number`, `tier_1`, `pwt_lbs`, `base_year_month`, `base_price`, `forecast_generated_at`, `forecasts`.
- `ErrorResponse`: `error`, `detail`.

> `predicted_price_without_deadband` is a **required** field on `MonthForecast`. Every place that builds a `MonthForecast` (currently only `forecast_engine.py`) must pass it, otherwise validation fails.

`app/api/v1/router.py`

Mounts the two active routers under `/api/v1`: `forecast_excel_router` and `forecast_batch_router`.

`app/api/v1/endpoints/forecast_excel.py`

Method	Path	Purpose
`GET`	`/api/v1/forecast-excel/parts`	List all parts with weight, price and price month
`POST`	`/api/v1/forecast-excel/reload`	Clear the Excel cache
`POST`	`/api/v1/forecast-excel`	Single-part forecast (breakdown included)

Errors: `404` for "Part not found", `422` for other value errors (missing price or market data, price-month mismatch), `500` for the Excel file missing or an unexpected failure. Repositories are wired in with `Depends`.

`app/api/v1/endpoints/forecast_batch.py`

`POST /api/v1/forecast-batch` (multipart form):

- `file`: `.xlsx` with `Part Number` and `Tier 1` (and optionally `CNG_Q` / `CNG_Q-1` per row, if the endpoint routes to the per-row path)
- `cng_q`, `cng_q-1`: required numbers on the shared-CNG path
- `include_current_month`: `YES` / `NO` (default `NO`)

Returns a `.xlsx` download named `forecast_<YYYY-MM-DD>.xlsx` with `Content-Disposition` and an `X-Parts-Count` header. `400` is returned for a non-.xlsx file, a bad `include_current_month`, missing columns, an empty list, or when all parts fail. This file was not part of this review — confirm against the current source whether it calls `build_forecast_workbook` (shared CNG) or `build_forecast_workbook_from_rows` (per-row CNG), and whether a preview route exists for `build_preview_from_rows`.

---

7. Data Layer

`aluminium_data.xlsx` : sheets

Sheet	Columns	Notes
Parts	Part Number, Tier 1, Weight (lbs), Current Price ($) *with month in header*	Key = Part Number + Tier 1
LME	Month (`YYYY-MM`), value[, Accuracy (%) in row 2]	$/lb. Can be auto-refreshed from `lme_input` (§3)
Midwest	Month (`YYYY-MM`), value	$/lb
PPI	Month (`YYYY-MM`), value[, Accuracy (%) in row 2]	Index value, not dollars. Can be auto-refreshed from `ppi_input` (§3)
lme_input (NEW, optional)	Date, Value	Monthly LME actuals, $/MT. Model input only — the engine never reads this sheet directly
ppi_input (NEW, optional)	Date, Value	Monthly PPI actuals. Model input only — the engine never reads this sheet directly
AOP (NEW, optional)	Part Number, Tier 1, one column per month (header names the month)	Planned quantities for Total AOP (§4). Sheet name/layout confirmed from the lookup code, not from `excel_store.py` directly

After editing the workbook, restart the server or call `POST /api/v1/forecast-excel/reload`. Editing `lme_input` / `ppi_input` does not need a manual reload for the batch endpoint — `refresh_lme_forecast()` / `refresh_ppi_forecast()` pick up the change automatically on the next batch run (§3.3) and invalidate the cache themselves.

Coverage needed: each forecast month needs LME, Midwest and PPI for itself, and for the previous quarter (the last month, or all three months when that quarter is not yet covered by actuals). Cover roughly two quarters back to 12 months ahead. If `lme_input` / `ppi_input` are present and cover enough history (at least ~24–36 months, more for a reliable backtest), the `LME` / `PPI` sheets' forward coverage is maintained automatically.

---

8. API Endpoints

Base URL: `http://localhost:8000`. Swagger UI at `/docs`, ReDoc at `/redoc`.

Method	Endpoint	Input	Output
`GET`	`/health`	none	`{"status":"ok","version":"1.0.0"}`
`GET`	`/api/v1/forecast-excel/parts`	none	Parts list
`POST`	`/api/v1/forecast-excel/reload`	none	Cache cleared
`POST`	`/api/v1/forecast-excel`	JSON body	JSON forecast
`POST`	`/api/v1/forecast-batch`	multipart form	`.xlsx` download

Single-part response (abbreviated)


```
{
  "part_number": "09-0052-003",
  "tier_1": "NA",
  "pwt_lbs": 3.6,
  "base_year_month": "2026-09",
  "base_price": 47.09,
  "forecast_generated_at": "2026-09-10T08:00:00+00:00",
  "forecasts": [
    {
      "year_month": "2026-10",
      "month_label": "October 2026",
      "predicted_price": 0.0,
      "predicted_price_without_deadband": 0.0,
      "base_price_used": 47.09,
      "pwt": 3.6,
      "df_c": 1.44,
      "quarter_context": {
        "quarter_label": "Q4-2026",
        "mc_q": 0.0, "lme_q": 0.0, "midwest_q": 0.0, "ppi_q": 0.0,
        "cng_q": 0.97, "ams_q": 0.0,
        "prev_quarter_label": "Q3-2026",
        "mc_q-1": 0.0, "lme_q-1": 0.0, "midwest_q-1": 0.0, "ppi_q-1": 0.0,
        "cng_q-1": 0.95, "ams_q-1": 0.0,
        "ppi_factor": 0.0, "ams_delta": 0.0
      },
      "price_change_breakdown": { "...": "lme/midwest/cng/ppi effects + sum_check" },
      "is_data_projected": false
    }
  ]
}
```

Values shown as `0.0` are placeholders. Real numbers depend on the data in the workbook.

---

9. Getting Started

Prerequisites

- Python 3.12 recommended
- `aluminium_data.xlsx` next to `main.py` (or set `ALUMINIUM_EXCEL_PATH`)

Install

```
python -m venv env
env\Scripts\activate          # Windows
source env/bin/activate       # Mac / Linux
pip install -r requirements.txt
```

`requirements.txt` (update needed)

```
fastapi>=0.115.0
uvicorn[standard]>=0.30.6
pydantic>=2.11.0
pydantic-settings>=2.7.0
python-dateutil>=2.9.0
openpyxl>=3.1.0
pandas>=2.0.0
python-multipart>=0.0.9
numpy>=1.26.0           # NEW - used by lme_forecast.py / ppi_forecast.py
statsmodels>=0.14.0      # NEW - SARIMAX model for LME/PPI auto-forecasting
joblib>=1.5.3
scikit-learn>=1.9.0
```

`joblib` and `scikit-learn` are not imported by any current code (left over from a removed ML endpoint) and can be dropped. `numpy` and `statsmodels` are **new requirements** for §3 — without them, `refresh_lme_forecast()` / `refresh_ppi_forecast()` will fail (logged, not raised) and the batch will silently fall back to whatever is already in the `LME` / `PPI` sheets.

Run / test

```
uvicorn main:app --reload --port 8000
pytest tests/ -v          # see §13: tests need updating
```

Optional: custom Excel path

```
export ALUMINIUM_EXCEL_PATH=/full/path/to/aluminium_data.xlsx      # Mac / Linux
set ALUMINIUM_EXCEL_PATH=C:\full\path\to\aluminium_data.xlsx       # Windows
```

---

10. How to Swap a Data Source

The engine depends only on the abstract classes in `base.py`.

Live market data (BLS, LME feed): create a class inheriting `MarketDataRepository` that implements `get_lme`, `get_midwest_premium` and `get_ppi`, then change the factory in the endpoint files:

```
def get_market_repo() -> MarketDataRepository:
    return LiveApiMarketDataRepository()
```

ERP for parts: implement `PartRepository` (`get_part_weight`, `get_base_price`, optionally `get_price_month`) and change `get_part_repo()`.

Note the factories are duplicated in `forecast_excel.py` and `forecast_batch.py`, so change both. The LME/PPI auto-forecast (§3) is independent of this — it reads/writes the Excel workbook directly via `openpyxl` / `pandas`, not through `MarketDataRepository`, so swapping the market data source does not automatically carry the auto-forecast logic with it.

---

11. Excel File Format

Parts sheet

Part Number	Tier 1	Weight (lbs)	Current Price ($) August 2026
09-0052-003	NA	3.6	47.09
2190-1015	NA	7	89.97

- **Part Number + Tier 1 = unique key**. `NA` means no specific supplier.
- Matching is exact apart from surrounding whitespace.
- The price column header **must name its month**. Update the prices and the header together.
  - Header month = last month → use `include_current_month: "YES"`.
  - Header month = this month → use `include_current_month: "NO"`.
- Price cells that contain currency text such as `$1,233.07` must be stored as plain numbers, because values are cast with `float()`.

LME, Midwest, PPI sheets

Month (YYYY-MM)	Value
2026-07	1.4751

If LME/PPI auto-forecasting is in use (§3), an `Accuracy (%)` column is added automatically in row 2 and kept up to date — no manual edits needed there.

lme_input / ppi_input sheets (optional, drive §3)

Date	Value
2025-07	3260.40

- `lme_input` values are in **$/MT** (converted to $/lb when written to the `LME` sheet). `ppi_input` values are the PPI index directly.
- Dates can be in almost any common format (see §3.4) — Excel date cells, `2026-10`, `Oct 2026`, `Oct'26`, `10/2026`, etc.
- Must have **no gaps** in the month sequence — a missing month raises an error (logged, batch still continues with the existing `LME` / `PPI` sheet).

AOP sheet (optional, drives §4)

Part Number	Tier 1	Oct 2026	Nov 2026	...
221134	NA	1200	1150	...

- Blank or non-numeric quantity cells are skipped (treated as "no AOP for that part/month").
- A row with a blank Tier 1 acts as an item-level fallback used when no tier-specific quantity is found for that part/month.

CNG is **not** stored in the main `Parts` / `LME` / `Midwest` / `PPI` sheets. It is either supplied once in the request/form (shared-CNG path) or read per row from the **batch input file** (`CNG_Q`, `CNG_Q-1` columns, several header spellings accepted — see §6, `read_parts_with_cng_from_upload`).

> The sample `aluminium_data.xlsx` and `input.xlsx` in the archive are tab-separated text files, not real Excel workbooks. The sample `aluminium_data.xlsx` also contains only the Parts data. Re-save them as genuine `.xlsx` files with all the sheets above before running the service.

---

12. Frontend Integration

Single-part forecast (Axios)

```
const { data } = await axios.post('/api/v1/forecast-excel', {
  part_number: '09-0052-003',
  tier_1: 'NA',
  cng_q: 0.97,
  'cng_q-1': 0.95,
  include_current_month: 'NO'
});

// data.forecasts[0].predicted_price                    <- final, with-deadband
price
// data.forecasts[0].predicted_price_without_deadband     <- raw price before the
deadband
// data.forecasts[0].base_price_used
// data.forecasts[0].price_change_breakdown
```

Batch forecast (Axios)

```
const form = new FormData();
form.append('file', excelFile);     // Part Number, Tier 1[, CNG_Q, CNG_Q-1 per row]
form.append('cng_q', '0.97');       // only needed on the shared-CNG path
form.append('cng_q-1', '0.95');
form.append('include_current_month', 'NO');

const response = await axios.post('/api/v1/forecast-batch', form, {
  responseType: 'blob'
});

const url = URL.createObjectURL(response.data);
const a = document.createElement('a');
a.href = url;
a.download = 'forecast_result.xlsx';
a.click();
URL.revokeObjectURL(url);
```

Reading the downloaded workbook: the monthly sheets' "**Predicted Price**" column is the *raw* (pre-deadband) value and "**Final Unit Price (with deadband)**" is the actual final price — the reverse of the JSON field naming (§2.4, §6). If a frontend needs to show "the price", use "**Final Unit Price (with deadband)**", not "**Predicted Price**", from the Excel output. The **Summary** and **Base Price Summary** sheets are unaffected by this naming quirk — they're already named for what they contain.

Batch input format

Columns `Part Number` and `Tier 1` (case-insensitive) are always required. Optional `CNG_Q` and `CNG_Q-1` columns (several spellings accepted) let each row carry its own CNG values instead of one

shared pair for the whole file. Duplicate pairs are de-duplicated silently, and blank rows are skipped.

---

13. Known Gaps and Legacy Files

- `approach.txt` describes an **earlier design**: the price is the same for every month of a quarter, CNG is fixed at 0.97, `MC = (LME + Midwest) × 0.91 + 1.25`, and `PPI_Q` is a quarter average. The code now differs on all of these (see §2): the base price moves only in the last month of each quarter (and each month still gets its own calculated price), CNG comes from the request or the upload file, MC has no multiplier, and `PPI_Q` is the predicted month's value.
- `forecast.py` is a legacy endpoint that is **not registered** in the router. It still passes `current_price` and uses the hardcoded repositories, so it would fail against the current request model and engine signature.
- `test_forecast_engine.py` targets an older engine shape entirely: it calls `forecast()` without CNG arguments, expects forecasts starting March 2026, and assumes chained prices. It now also does not cover the quarterly base-price rule, the deadband, AOP, or the LME/PPI auto-forecasting — all of these currently ship untested.
- **Docstrings** in `response.py` (`QuarterContext.mc_q` / `mc_q_1`) still mention the `× 0.91 + 1.25` MC formula. This comment is out of date; the engine uses `+ MC_OFFSET` only.
- `excel_store.py` reads Parts columns by position, so keep the Parts column order unchanged.
- **Excel column naming swap (§2.4, §6, §12):** in the batch output, "Predicted Price" shows the raw, pre-deadband value, and "Final Unit Price (with deadband)" shows the actual final price. This is intentional per the latest requirements but easy to misread, since it's the reverse of the JSON API's `predicted_price` naming. Worth double-checking with downstream consumers of the Excel file.
- **In-place writes to the workbook (§3.4):** a successful LME/PPI refresh modifies `aluminium_data.xlsx` on disk during a batch run. There's no cross-process file lock — if two batch requests (or a batch request and someone editing the file in Excel) overlap, a `PermissionError` or a lost update is possible. `ppi_forecast.py` takes a backup first; `lme_forecast.py` currently does not.
- **AOP sheet contract is not validated end-to-end here:** `excel_store.py` (which defines `_load_aop_lookup()`) was not part of this review, so the exact AOP sheet name and header parsing should be double-checked against that file directly, especially if Total AOP values come back unexpectedly blank.
- `forecast_batch.py` **endpoint file** (the actual `POST /api/v1/forecast-batch` route handler) was not part of this review either — confirm there which of `build_forecast_workbook` / `build_forecast_workbook_from_rows` / `build_preview_from_rows` it actually calls, and under what conditions it picks the shared-CNG vs. per-row-CNG path.

For questions about the formula or data sources, refer to the quarterly pricing agreement documentation or contact the pricing team.